

Slice Phase 8 → 9 Handoff

1. Objective of Phase 9

Phase 9 is defined in the architecture as “Evaluation, Testing & Refinement.” The goal is not to add new conceptual features but to validate and harden everything built in Phases 1–8. For Slice as it exists at the end of Phase 8, this means:

- Verifying that end-to-end flows from data → repositories → context builders → API/UI behave correctly with realistic data.
- Tightening contracts between the quantitative layer, risk interface, memory/LLM layer, and the UI.
- Identifying and fixing bugs, edge cases, and misalignments with the original design document.
- Evaluating whether the system, as implemented, actually supports discretionary macro thesis development in the way the roadmap intended.

2. Current Surface Area at End of Phase 8

The Phase 9 work starts from the concrete code and behavior described in the Phase 8 completion:

- Data & Quant:
 - Data ingestion, repositories, and the quant sandbox from earlier phases (price/macro series, basic backtesting and metrics) exist and are callable from Python.
 - A minimal risk interface (RiskSnapshot + get_snapshot) exposes a stable, JSON-serializable structure even though the internal risk math is intentionally minimal.
- Intuition & LLM:
 - Observation ingestion, embedding, and recall are implemented under the memory app and associated repositories.
 - LLM-based tools for thesis review, consistency checking, intuition Q&A, and daily summaries exist as callable functions or API endpoints.
- API Layer:
 - FastAPI main app in src/slice/api/main.py with /session, /intelligence, and /memory surfaces.
- UI Layer:
 - Streamlit app in ui/app.py with tabs that render portfolio, strategy, diagnostics, and narrative contexts using DataAccess.
 - All tabs currently operate in read-only mode and correctly display placeholder contexts when DB is empty.

Phase 9 must treat these components as the baseline and avoid breaking their public contracts without deliberate, coordinated changes to tests and prompts.

3. End-to-End Flows to Validate

Phase 9 work should be organized around concrete end-to-end flows. Each flow should be tested from the top-level entrypoint down to repositories and back again.

A. Portfolio Snapshot Flow

- Entry: UI Portfolio tab → load_portfolio_view() → DataAccess.get_portfolio_context().
- Dependencies: PortfolioRepository, TradeRepository (for positions), optional risk interface.
- Expected behavior:
 - With no positions: positions = [], totals all zeros, factor_exposures aggregate betas = 0, thesis_exposures empty.

- With sample positions inserted: positions list should be populated, totals should reflect correct book value and exposures by sign, while factor_exposures may still be stubbed.
- No exceptions are allowed; errors must be converted into sane defaults.

B. Strategy Context Flow

- Entry: UI Strategy Context tab → `DataAccess.get_strategy_context()`.
- Dependencies: ThesisRepository, portfolio context, macro data utilities.
- Expected behavior:
 - active_theses contains structured theses when they exist; otherwise [].
 - macro_view.macro_snapshot contains current macro metrics when data is available; otherwise null/None.
 - regimes keys (growth, inflation, liquidity, usd) are always present; values are one of the agreed labels (currently "unknown" in the placeholder implementation).

C. Portfolio Diagnostics Flow

- Entry: UI Diagnostics tab → `DataAccess.get_portfolio_diagnostics_context()`.
- Dependencies: Same as portfolio context plus scenario utilities.
- Expected behavior:
 - stress_tests always returns a list of scenario objects with scenario names and impact fields (None until real stress logic is implemented).
 - thesis_exposures and factor_exposures align structurally with portfolio context.

D. Narrative Context Flow

- Entry: UI Narrative tab → `DataAccess.get_narrative_context()`.
- Dependencies: ThesisRepository, macro utilities, quant summaries.
- Expected behavior:
 - The narrative_context payload is always well-formed, with explicit keys for theses, macro_view, portfolio_snapshot, quant_summaries, and commentary_window.
 - This context should be directly usable as the "world state" for Phase-6/7 narrative prompts.

E. Memory and LLM Flows

- Entry: /memory endpoints and /intelligence endpoints for thesis review, consistency checks, intuition Q&A.
- Phase 9 should:
 - Inject realistic sample theses and observations.
 - Run through the LLM workflows using the live contexts.
 - Confirm that no endpoint assumes fields that DataAccess does not populate.

4. Test and Evaluation Plan for Phase 9

Phase 9 should introduce a layered test strategy:

1) Unit-Level Tests

- Add tests for DataAccess methods:
 - Verify that each context builder returns dictionaries with the expected top-level keys.
 - Validate type shapes (e.g. positions is a list, totals is a dict with specific keys, regimes is a dict).
 - Confirm that methods handle missing or malformed data from repositories by returning defaults, not raising.
- Add tests for risk.interface.get_snapshot:
 - Case with no trades: returns None.
 - Case with mocked TradeRepository returning trades: returns a RiskSnapshot with zeroed values, never raises.
 - render_risk_snapshot_text produces deterministic output given a known snapshot.

2) Integration Tests

- Use pytest + a temporary database to:
 - Insert sample theses, positions, and trades via repositories.
 - Call DataAccess context builders and assert that sample data is visible in the returned structures.
 - Hit FastAPI endpoints for /session and /intelligence with httpx or TestClient to ensure correct HTTP status codes and JSON shapes.

3) UI Smoke Tests

- At minimum, import ui/app.py in a test to confirm that all imports succeed and that no top-level code throws.
- Optionally use Streamlit's test utilities or a subprocess runner to ensure streamlit run ui/app.py exits cleanly in a headless mode.

4) LLM Prompt Evaluation

- For thesis review, consistency checker, intuition Q&A, and summaries:
 - Construct a small fixed corpus of sample theses and observations.
 - Capture LLM responses and check for:
 - Presence of required JSON keys where function calling or JSON output is expected.
 - Absence of hallucinated internal fields (the LLM should not invent schema keys).
 - Qualitative quality of questions and commentary.
 - Adjust prompts only when tests show systematic problems, not for cosmetic reasons.

5. Known Gaps and Guardrails for Phase 9

Phase 9 should refine and harden the system without turning into a stealth feature phase. The following constraints apply:

- Risk math is intentionally incomplete:
 - Phase 9 may improve internal calculations behind RiskSnapshot (e.g. use real trade data for book_gross/book_net), but must not change the public schema or the function signature of get_snapshot or render_risk_snapshot_text.
 - Any future, richer risk engine should plug into this interface, not replace it.
- Context schemas are contracts:
 - The shapes of portfolio_context, strategy_context, portfolio_diagnostics_context, and narrative_context are de facto API contracts for both the UI and LLM prompts.
 - Phase 9 can add fields but should not rename or remove existing ones without also updating prompts, tests, and documentation in a coordinated way.
- UI scope is fixed for Phase 9:
 - The Streamlit app remains a shell for viewing context and, at most, for triggering analyses.
 - No trade execution buttons, broker integration, or complex new UI flows should be added in Phase 9.
- No regime/driver re-architecture:
 - Phase 9 should treat macro_view and regimes as given. Any change to how regimes are defined or labeled belongs in a separate architectural phase to avoid invalidating earlier work.

These guardrails keep Phase 9 focused on reliability, not scope expansion.

6. Practical Instructions for the Phase 9 GPT Instance

When Phase 9 is driven by a new GPT instance, it should operate with the following concrete rules:

A. Code Navigation

- Primary locations to inspect and modify:
 - `src/slice/intelligence/context/data_access.py` – context construction contracts.
 - `src/slice/risk/interface.py` – risk snapshot interface and text rendering.
 - `src/slice/api/` – FastAPI routers and endpoint wiring.
 - `ui/app.py` – Streamlit surface, including how contexts are displayed.
 - `tests/` – where new unit and integration tests must be added.

B. Invariants to Preserve

- Import patterns must remain consistent with the current working app (no reintroduction of erroneous `"from src.slice..."` imports under Streamlit).
- Public function signatures used by other modules (e.g. `DataAccess` methods, `get_snapshot`) must not change.
- Contexts must continue to be fully JSON-serializable (no non-serializable objects like `datetimes` or `Pydantic` models leaking through).

C. Test and Run Commands

- To run tests:

```
cd ~/dev/slice
PYTHONPATH=src pytest
```
- To run the API locally (for manual inspection or future integration tests):

```
uvicorn slice.api.main:app --reload
```
- To run the UI locally:

```
streamlit run ui/app.py
```

The Phase 9 GPT instance should assume these commands are the canonical way to validate its changes.

7. Definition of Done for Phase 9

Phase 9 is complete when the following are all true:

- All context builders and risk interfaces are covered by unit tests that assert schema shape and error-handling behavior.
- Integration tests confirm that FastAPI endpoints and the UI can be started against a temporary database with sample data, without crashes.
- LLM tools (thesis review, consistency check, intuition Q&A, summaries) have been run on a fixed set of sample data and their outputs have been qualitatively evaluated and iteratively improved.
- No regressions are introduced into the contracts or behaviors documented in the Phase 8 completion document.
- The PM can use the system for a short live trial (even with limited real data) without encountering stability issues, and feels that the outputs are coherent with the design intent.

Once these conditions hold, Slice's v1 architecture - from data to quant to intuition to UI - can be considered stable, and subsequent work can focus on richer risk math, strategy libraries, or broker integration rather than foundational plumbing.